ackee
blockchain security

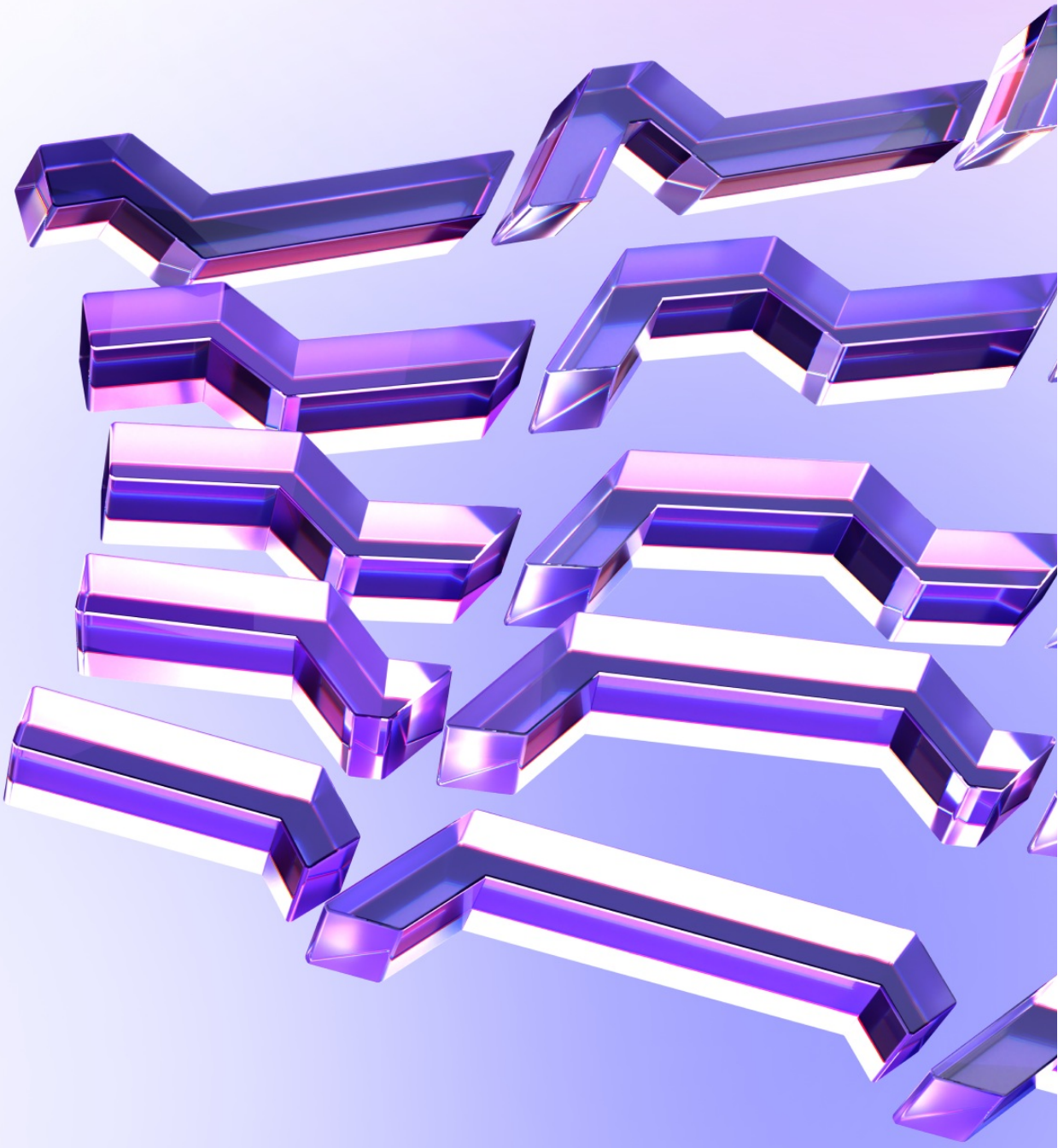
Wake

CMTA

CMTAT

February 10, 2026 12:24 UTC

Wake Arena AI Report



Contents

1. Document Revisions . . . 3
2. Overview . . . 4
2.1. Ackee Blockchain Security . . . 4
2.2. Wake Arena . . . 5
2.3. Disclaimer . . . 5
2.4. Finding Classification . . . 6
3. Executive Summary . . . 7
Revision 1.0 . . . 7
4. Findings Summary . . . 10
Report Revision 1.0 . . . 12
Findings . . . 12
Appendix A: How to cite . . . 32

1. Document Revisions

1.0	AI Scan	Feb 10, 2026 12:24 UTC

2. Overview

This document presents the findings Wake Arena identified using AI vulnerability checks in the reviewed contracts.

2.1. Ackee Blockchain Security

Ackee Blockchain Security is an in-house team of security researchers performing security audits focusing on manual code reviews with extensive fuzz testing for Ethereum and Solana. Ackee is trusted by top-tier organizations in web3, securing protocols including Lido, Safe, and Axelar.

We develop open-source security and developer tooling Wake for Ethereum and Trident for Solana, supported by grants from Coinbase and the Solana Foundation. Wake and Trident help auditors in the manual review process to discover hardly recognizable edge-case vulnerabilities.

Our team teaches about blockchain security at the Czech Technical University in Prague, led by our co-founder and CEO, Josef Gattermayer, Ph.D. As the official educational partners of the Solana Foundation, we run the School of Solana and the Solana Auditors Bootcamp.

Ackee's mission is to build a stronger blockchain community by sharing our knowledge.

Ackee Blockchain a.s.
Rohanske nabrezi 717/4
186 00 Prague, Czech Republic
https://ackee.xyz
hello@ackee.xyz

2.2. Wake Arena

This report was created by Wake Arena, an automated vulnerability analysis tool. It uses Wake Framework with additional detectors for AI and static analysis of Solidity smart contracts.

To find potential vulnerabilities, Wake uses:

- The code structure and patterns
- Control flow graphs
- Data flow graphs
- Common vulnerability patterns
- Contract interactions

Wake Arena aims for precision, not recall. This means the presented findings are optimized for true positives, not for finding all possible issues, and should always be complemented with additional manual review for a complete security assessment.

2.3. Disclaimer

We've put our best effort to find known vulnerabilities in the system, but automated findings shouldn't be considered a complete list of all existing issues. The statements made in this document should not be interpreted as investment or legal advice, nor should its authors be held accountable for decisions made based on them.

2.4. Finding Classification

Each finding is classified by two independent ratings: *Impact* and *Confidence*.

Impact

Measuring the potential consequences of the issue on the system.

- **Critical** - Code that activates the issue directly enables theft of significant assets with minimal preconditions.
- **High** - Code that activates the issue will lead to undefined or catastrophic consequences for the system.
- **Medium** - Code that activates the issue will result in consequences of serious substance.
- **Low** - Code that activates the issue will have outcomes on the system that are either recoverable or don't jeopardize its regular functioning.
- **Warning** - The issue represents a potential security concern in the code structure or logic that could become problematic with code modifications.
- **Info** - The issue relates to code quality practices that may affect security. Examples include insufficient logging for critical operations or inconsistent error handling patterns.

Confidence

Indicating the probability that the identified issue is a valid security concern.

- **High** - The analysis has identified a pattern that strongly indicates the presence of the issue.
- **Medium** - Evidence suggests the issue exists, but manual verification is recommended.
- **Low** - Potential indicators of the issue have been detected, but there is a significant possibility of false positives.

3. Executive Summary

Revision 1.0

Audit Overview

This executive summary covers the CMTA/CMTAT repository at commit `b6e9e3ca78f76d42f7f04b62a876eb6b081e5d75`, focusing on the upgradeable token architecture that composes OpenZeppelin Upgradeable primitives with modular validation and rule-engine layers, ERC2771 meta-transaction support, ERC1404-style restriction surfaces, and a cross-chain aware ERC20 module.

Across the scoped deployments and modules (CMTATUpgradeable, CMTATUpgradeableAllowlist, CMTATStandaloneLight, CMTATUpgradeableLight), 6 AI-detected issues were identified (0 critical, 0 high, 3 medium, 2 low, 1 info). The most severe cluster of issues involves redundant execution of the external compliance rule-engine hook, which can double-charge quotas, skew counters, and cause avoidable reverts:

- In `contracts/modules/4_CMTATBaseERC20CrossChain.sol`, functions `_mintOverride`, `_burnOverride`, and `_minterTransferOverride` call `_checkTransferred` and then delegate to `CMTATBaseCommon`, which performs the same check again, resulting in two identical `IRuleEngine.transferred` executions per operation. This creates inconsistent compliance behavior and inflates gas with potential denial-of-service on otherwise valid mints, burns, and minter-initiated transfers.

Additional findings include the well-known ERC20 allowance change race exposure in the allowlist variant, a log/state divergence for the custom `Spend` event when allowances are infinite, and a documentation mismatch around burner roles. Overall, the codebase exhibits a strong security posture: it leverages audited OpenZeppelin libraries, applies role-based access control and pausability, and cleanly separates validation concerns. The medium-

severity correctness issues around rule-engine integration are specific and straightforward to remediate and should be addressed before production deployment.

Key Technical Findings

- Duplicate compliance hook execution across cross-chain overrides: `CMTATBaseERC20CrossChain._mintOverride`, `_burnOverride`, and `_minterTransferOverride` each invoke `CMTATBaseRuleEngine._checkTransferred(…)` and then delegate to `CMTATBaseCommon` which invokes the same check again, resulting in two identical `IRuleEngine.transferred` calls per operation (file: `contracts/modules/4_CMTATBaseERC20CrossChain.sol`). Remediation: remove the redundant call in the overrides and rely on the single authoritative call in `CMTATBaseCommon`.
- ERC20 approve race exposure in allowlist variant: `CMTATBaseAllowlist.approve` (file: `contracts/modules/2_CMTATBaseAllowlist.sol:131-136`) preserves the standard allowance change race and can be front-run by the spender when changing nonzero allowances. Remediation: enforce zero-first updates in `approve` or require the `increaseAllowance`/`decreaseAllowance` pattern.
- Misleading allowance accounting event: `ERC20BaseModule.transferFrom` (file: `contracts/modules/wrapper/core/ERC20BaseModule.sol`) unconditionally emits `Spend(account, spender, value)` even when allowance is infinite and no deduction occurs per OpenZeppelin `_spendAllowance`. Remediation: emit `Spend` only when a finite allowance is actually reduced, or emit the computed delta.
- Documentation inconsistency: in `CMTATBaseERC20CrossChain._authorizeSelfBurn` (file: `contracts/modules/4_CMTATBaseERC20CrossChain.sol:219-230`), the comment

references `BURNER_FROM_ROLE` while the code enforces `BURNER_SELF_ROLE`. Remediation: align the comment and add tests asserting the correct role gate.

Conclusion

The audit identified no critical or high issues and a focused set of medium-severity correctness problems in the compliance hook integration, alongside low-impact allowance/event semantics and a minor documentation issue. With the duplicate hook invocations resolved, allowance update policy hardened, and event emission semantics aligned with on-chain state, the codebase appears on track for deployment. We recommend addressing the medium findings before mainnet and verifying through targeted unit tests that each high-level operation triggers the rule-engine exactly once.

4. Findings Summary

Summary of findings:

Critical	High	Medium	Low	Warning	Info	Total
0	0	3	2	0	1	**6**

Table 1. Findings Count by Impact

Findings in detail:

Finding title	Impact	Reported	Status
M1: Double invocation of rule-engine compliance hook in `_minterTransferOverride` triggers duplicate side effects	Medium	1.0	Reported
M2: Double invocation of rule-engine compliance hook in `_burnOverride` triggers duplicate side effects	Medium	1.0	Reported
M3: Double invocation of rule-engine compliance hook in `_mintOverride` triggers duplicate side effects	Medium	1.0	Reported
L1: Misleading Spend event emitted on transferFrom when allowance is infinite	Low	1.0	Reported
L2: Unmitigated ERC20 approve allowance change can be front-run by spender	Low	1.0	Reported

Finding title	Impact	Reported	Status
I1: Documentation mismatch: `_authorizeSelfBurn` comment references `BURNER_FROM_ROLE` but code enforces `BURNER_SELF_ROLE`	Info	1.0	Reported

Table 2. Table of Findings

Report Revision 1.0

Findings

The following section presents the list of findings discovered in this revision.

For the complete list of all findings, Go back to Findings Summary

M1: Double invocation of rule-engine compliance hook in `_minterTransferOverride` triggers duplicate side effects

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/modules/4_CMTATBaseERC20CrossChain.sol	Type:	Logic error

Description

The override `CMTATBaseERC20CrossChain._minterTransferOverride` calls `CMTATBaseRuleEngine._checkTransferred(address(0), from, to, value)` and then delegates to `CMTATBaseCommon._minterTransferOverride(from, to, value)`, which itself invokes `_checkTransferred(address(0), from, to, value)` again before performing the transfer.

This leads to the external rule engine `transferred` hook being executed twice for a single minter-initiated transfer, which can skew quotas, cause unexpected reverts, or waste gas.

Affected components:

- Function `CMTATBaseERC20CrossChain._minterTransferOverride` in `contracts/modules/4_CMTATBaseERC20CrossChain.sol`.
- Function `CMTATBaseCommon._minterTransferOverride` in `contracts/modules/0_CMTATBaseCommon.sol` (calls `_checkTransferred` again).
- Function `CMTATBaseRuleEngine._checkTransferred` in `contracts/modules/2_CMTATBaseRuleEngine.sol` (forwards to `ValidationModuleRuleEngine._transferred`).

Listing 1. Code snippet from contracts/modules/4_CMTATBaseERC20CrossChain.sol

```
    function _minterTransferOverride(
        address from,
        address to,
        uint256 value
    ) internal virtual override(CMTATBaseCommon, ERC20MintModuleInternal) {
        CMTATBaseRuleEngine._checkTransferred(address(0), from, to, value);
        CMTATBaseCommon._minterTransferOverride(from, to, value);
    }
```

Exploit Scenario

A minter-initiated transfer that should pass a single compliance check can be made to fail or exhaust quotas due to double hook execution.

Consider the following attack scenario:

1. Alice, a privileged minter, initiates a transfer from `from` to `to` once;
2. The contract executes `_checkTransferred` twice for the same transfer (once directly and once via `CMTATBaseCommon._minterTransferOverride`), resulting in two `transferred` hook executions; and
3. Bob, the recipient, experiences a revert or sees quotas exhausted unexpectedly, degrading protocol UX and reliability.

Recommendation

Remove the duplicate validation call so the rule engine hook is executed exactly once per minter-initiated transfer.

Recommended approach:

- Delete the direct call to `CMTATBaseRuleEngine._checkTransferred` from `CMTATBaseERC20CrossChain._minterTransferOverride` and rely on `CMTATBaseCommon._minterTransferOverride` to perform the single

authoritative check/hook; or

- If this override needs extra pre-checks, call only the internal transfer implementation after those checks and avoid calling `_checkTransferred` twice.

Validation guidance:

- Add a test stub `IRuleEngine` that counts invocations of `transferred` and assert a single call per minter transfer.

Go back to Findings Summary

M2: Double invocation of rule-engine compliance hook in `_burnOverride` triggers duplicate side effects

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/modules/4_CMTATBaseERC20CrossChain.sol	Type:	Logic error

Description

The override `CMTATBaseERC20CrossChain._burnOverride` redundantly traverses the compliance path twice:

- First, via `CMTATBaseRuleEngine._checkTransferred(address(0), account, address(0), value)`, which forwards to `ValidationModuleRuleEngine._transferred` and calls the external `IRuleEngine` hook; and
- Second, by delegating to `CMTATBaseCommon._burnOverride(account, value)`, which invokes `_checkTransferred(address(0), account, address(0), value)` again before burning.

A single burn therefore causes the external rule engine `transferred` hook to run twice with identical arguments. Engines that mutate state or enforce one-call-per-operation invariants can double-count or revert, and gas usage is inflated unnecessarily.

Affected components:

- Function `CMTATBaseERC20CrossChain._burnOverride` in `contracts/modules/4_CMTATBaseERC20CrossChain.sol`.

- Function `CMTATBaseCommon._burnOverride` in `contracts/modules/0_CMTATBaseCommon.sol` (calls `_checkTransferred` again).
- Function `CMTATBaseRuleEngine._checkTransferred` in `contracts/modules/2_CMTATBaseRuleEngine.sol` (forwards to `ValidationModuleRuleEngine._transferred`).

Listing 2. Code snippet from contracts/modules/4_CMTATBaseERC20CrossChain.sol

```
    function _burnOverride(
        address account,
        uint256 value
    ) internal virtual override(CMTATBaseCommon, ERC20BurnModuleInternal) {
        CMTATBaseRuleEngine._checkTransferred(
            address(0),
            account,
            address(0),
            value
        );
        CMTATBaseCommon._burnOverride(account, value);
    }
```

Exploit Scenario

Rule engines that decrement allowances or track burn quotas can be driven into over-counting or premature reverts by the duplicate hook execution.

Consider the following attack scenario:

1. Alice holds tokens subject to a per-epoch burn quota enforced by the rule engine;
2. Alice triggers a burn that reaches `_burnOverride` once;
3. The contract executes `_checkTransferred` two times for the same burn (once directly and once via `CMTATBaseCommon._burnOverride`), causing the external rule engine `transferred` hook to be applied twice; and

4. Bob, the compliance operator, observes unexpected quota exhaustion or a revert, creating a denial of service for subsequent legitimate burns.

Recommendation

Remove the duplicate validation call to ensure exactly one hook execution per burn.

Recommended approach:

- Delete the direct call to `CMTATBaseRuleEngine._checkTransferred` from `CMTATBaseERC20CrossChain._burnOverride` and rely on `CMTATBaseCommon._burnOverride` to perform the single authoritative check/hook; or
- If pre-checks are needed in this override, invoke only the internal burn implementation after those checks (e.g., `ERC20BurnModuleInternal._burnOverride`) and avoid calling `_checkTransferred` twice.

Validation guidance:

- Add a test stub `IRuleEngine` that counts invocations of `transferred` and assert a single call per burn.

Go back to Findings Summary

M3: Double invocation of rule-engine compliance hook in `_mintOverride` triggers duplicate side effects

Medium impact finding

Impact:	Medium	Confidence:	High
Target:	contracts/modules/4_CMTATBaseERC20CrossChain.sol	Type:	Logic error

Description

The override `CMTATBaseERC20CrossChain._mintOverride` invokes the compliance path twice:

- First, by calling `CMTATBaseRuleEngine._checkTransferred(address(0), address(0), account, value)`, which forwards to `ValidationModuleRuleEngine._transferred` and calls the external `IRuleEngine` hook; and
- Second, by delegating to `CMTATBaseCommon._mintOverride(account, value)`, which itself calls `_checkTransferred(address(0), address(0), account, value)` again before minting.

This results in a single mint causing the external rule engine `transferred` hook to execute twice with identical arguments. Any rule engine that keeps counters, enforces quotas/rate limits, or mutates state upon `transferred` will exhibit double side effects, potentially reverting valid mints or producing inconsistent compliance/accounting. It also increases gas costs unnecessarily.

Affected components:

- Function `CMTATBaseERC20CrossChain._mintOverride` in `contracts/modules/4_CMTATBaseERC20CrossChain.sol`.
- Function `CMTATBaseCommon._mintOverride` in `contracts/modules/0_CMTATBaseCommon.sol` (calls `_checkTransferred` again).
- Function `CMTATBaseRuleEngine._checkTransferred` in `contracts/modules/2_CMTATBaseRuleEngine.sol` (forwards to `ValidationModuleRuleEngine._transferred`).

Listing 3. Code snippet from contracts/modules/4_CMTATBaseERC20CrossChain.sol

```
    function _mintOverride(
        address account,
        uint256 value
    ) internal virtual override(CMTATBaseCommon, ERC20MintModuleInternal) {
        CMTATBaseRuleEngine._checkTransferred(
            address(0),
            address(0),
            account,
            value
        );
        CMTATBaseCommon._mintOverride(account, value);
    }
```

Exploit Scenario

A rule engine that tracks transfer counts per unit time and reverts if a cap is exceeded can be coerced into denying legitimate mints due to double charging.

Consider the following attack scenario:

1. Alice, a privileged minter, configures the system with a rule engine that allows only one mint per epoch;
2. Alice calls the mint flow that reaches `_mintOverride` once;

3. The contract executes `_checkTransferred` two times for the same mint (once directly and once via `CMTATBaseCommon._mintOverride`), causing the external rule engine `transferred` hook to increment counters twice; and
4. Bob, the intended recipient, is now blocked from future mints in the same epoch, or the transaction reverts if the rule engine detects a second invocation, resulting in denial of service and unexpected compliance behavior.

Recommendation

Remove the redundant validation call in this override to ensure the rule engine hook executes exactly once per mint.

Recommended approach:

- Delete the direct call to `CMTATBaseRuleEngine._checkTransferred` from `CMTATBaseERC20CrossChain._mintOverride` and rely on `CMTATBaseCommon._mintOverride` to perform the single authoritative check/hook; or
- If specialized pre-checks are required in this override, call only the internal module mint function after those checks (e.g., `ERC20MintModuleInternal._mintOverride`) and avoid invoking `_checkTransferred` twice.

Validation guidance:

- Add a test stub `IRuleEngine` that counts invocations of `transferred` and assert a single call per mint.
- Consider an invariant test ensuring at most one `transferred` execution per high-level operation.

Go back to Findings Summary

L1: Misleading Spend event emitted on transferFrom when allowance is infinite

Low impact finding

Impact:	Low	Confidence:	High
Target:	./contracts/modules/wrapper/core/ERC20BaseModule.sol	Type:	Logging

Description

The module unconditionally emits the `Spend` event on every successful call to `transferFrom`, even when no allowance is actually reduced.

This contradicts both the documented semantics of event `Spend` and OpenZeppelin's allowance handling for the infinite-approval pattern, causing the event log to diverge from on-chain state and misleading off-chain systems that mirror allowance changes from events.

Listing 4. Code snippet from contracts/modules/wrapper/core/ERC20BaseModule.sol

```
function transferFrom(
    address from,
    address to,
    uint256 value
) public virtual override(ERC20Upgradeable) returns (bool) {
    bool result = ERC20Upgradeable.transferFrom(from, to, value);
    // The result will be normally always true because OpenZeppelin will revert in case of an error
    if (result) {
        emit Spend(from, _msgSender(), value);
    }

    return result;
}
```

The implementation above emits `Spend(from, spender, value)` for any successful transfer. However, OpenZeppelin's `_spendAllowance` deliberately skips decreasing the allowance if the current allowance equals `type(uint256).max` (i.e., infinite approval). In such cases, emitting `Spend` with `value` suggests a reduction that did not occur.

Listing 5. Code snippet from openzeppelin-contracts-upgradeable/contracts/token/ERC20/ERC20Upgradeable.sol

```
function _spendAllowance(
    address owner,
    address spender,
    uint256 value
) internal virtual {
    uint256 currentAllowance = allowance(owner, spender);
    if (currentAllowance < type(uint256).max) {
        if (currentAllowance < value) {
            revert ERC20InsufficientAllowance(
                spender,
                currentAllowance,
                value
            );
        }
        unchecked {
            _approve(owner, spender, currentAllowance - value, false);
        }
    }
}
```

The project interface `IERC20Allowance` explicitly states the `Spend` event "represents a reduction in the spender's allowance." Emitting it when no change occurs contradicts this contract-level specification and can corrupt off-chain mirrors that treat `Spend` as the single source of truth for allowance deltas.

Listing 6. Code snippet from contracts/interfaces/technical/IERC20Allowance.sol

```
/**
```

```
 * @notice Emitted when a `spender` spends a `value` amount of tokens on behalf of an `account`.
 * @dev
 * - This event is similar in semantics to the ERC-20 `Approval` event.
 * Approval(address indexed _owner, address indexed _spender, uint256 _value)
 * - It represents a reduction in the spender's allowance granted by the account.
 * - Can also be used for function which uses the allowance, e.g.`burnFrom
 * @param account The owner of the tokens whose allowance is being spent.
 * @param spender The address authorized to spend the tokens.
 * @param value The amount of tokens that were spent.
 */

event Spend(
    address indexed account,
    address indexed spender,
    uint256 value
);
```

Affected components:

- Function: `ERC20BaseModule.transferFrom`.
- Event: `IERC20Allowance.Spend`.
- State: OpenZeppelin `ERC20Upgradeable` allowances mapping; no decrease occurs in the infinite-approval case, creating a log/state mismatch.

Impact assessment:

- Off-chain indexers and accounting systems that decrement allowances based on `Spend` will record false reductions and may think allowance reaches zero when, on-chain, it remains infinite.
- This can lead to operational DoS (e.g., client UIs/bridges disabling actions due to perceived lack of allowance), inconsistent ledgers, or incorrect risk/limit management. There is no direct on-chain fund loss, but the discrepancy is material for downstream integrators.

Exploit Scenario

An on-chain adversary can reliably produce misleading allowance-change events without changing the actual allowance by leveraging infinite approvals.

Consider the following attack scenario:

1. Alice grants `spender` an unlimited allowance by calling `approve` with `type(uint256).max`.
2. Spender invokes `transferFrom` on Alice's balance to move `value` tokens repeatedly.
3. The contract processes each transfer successfully and emits `Spend(Alice, spender, value)` every time, while `allowance(Alice, spender)` remains `type(uint256).max`.
4. Bob's protocol, which mirrors allowances by consuming `Spend` events, incorrectly decrements Alice's stored allowance and eventually believes it is exhausted, triggering erroneous limits, blocking flows, or misreporting balances (operational DoS and accounting inconsistencies).

Recommendation

Align event emission with actual allowance reduction to avoid log/state divergence.

Recommended approaches:

- Emit `Spend` only when the allowance is finite. For example, read `allowance(from, _msgSender())` before the transfer; if it equals `type(uint256).max`, skip emitting `Spend`.
- Alternatively, compute and emit the actual amount reduced by reading the allowance before the parent `transferFrom`, calling the parent `transferFrom`, reading the allowance after, computing the difference (zero

when the previous allowance was infinite), and emitting `Spend` only if the computed difference is greater than zero.

- If changing semantics is desired, update the interface and downstream documentation to clarify that `Spend` does not necessarily imply an allowance reduction; however, this is discouraged because it breaks the current stated contract of the event.

Testing considerations:

- Add unit tests for both finite and infinite allowance paths asserting that `Spend` is emitted with the actual deducted amount and is skipped when no deduction occurs.

Go back to Findings Summary

L2: Unmitigated ERC20 approve allowance change can be front-run by spender

Low impact finding

Impact:	Low	Confidence:	High
Target:	contracts/modules/2_CMTATBaseAllowlist.sol	Type:	Front-running

Description

The contract preserves the well-known ERC20 allowance race condition. Function `approve` only adds `whenNotPaused` and delegates to OpenZeppelin `ERC20Upgradeable.approve` without enforcing a zero-first update or using the `increaseAllowance`/`decreaseAllowance` flow. If an owner changes an existing nonzero allowance for a spender, the spender can front-run the change and pull tokens at the old allowance before the new value is applied. Depending on relative ordering, the spender may be able to consume both the old and the new allowance, contrary to the owner's intent.

Affected elements:

- Function `approve` in contract `CMTATBaseAllowlist`;
- Underlying ERC20 storage `_allowances[owner][spender]` (from `ERC20Upgradeable`).

Impact in production:

- When owners reduce allowances, a malicious spender can race to drain up to the current allowance before the reduction confirms;
- When owners raise allowances, a malicious spender can consume the old allowance in a frontrun and still retain the new higher allowance afterward; and

- This can lead to more tokens being transferred than the owner intended within the same block or a short window, especially on high-MEV networks.

Listing 7. Code snippet from contracts/modules/2_CMTATBaseAllowlist.sol

```
function approve(
    address spender,
    uint256 value
) public virtual override(ERC20Upgradeable) whenNotPaused returns (bool) {
    return ERC20Upgradeable.approve(spender, value);
}
```

The implementation above does not include any mitigation such as requiring `value == 0 || allowance(_msgSender(), spender) == 0` or deprecating `approve` in favor of `increaseAllowance`/`decreaseAllowance`.

Exploit Scenario

A malicious spender can exploit the allowance change race by front-running an owner's `approve` update.

Consider the following attack scenario:

1. Bob, the token owner, has previously approved Alice for `N` tokens (`allowance(Bob, Alice) = N`);
2. Bob decides to change the allowance to `M` and submits `approve(Alice, M)`;
3. Alice observes the pending transaction and quickly submits `transferFrom(Bob, Alice, N)` (or up to `N`), with a higher gas price to be mined first;
4. Alice's `transferFrom` executes before Bob's `approve`, consuming the old allowance `N` and pulling tokens from Bob; and
5. Bob's `approve` then executes, setting the new allowance `M`. Alice now still has `M` allowance and can continue to pull additional tokens later, resulting in more tokens transferred than Bob intended.

Recommendation

Mitigate the ERC20 allowance race by enforcing a safe allowance update policy in `approve`.

Fix the vulnerability by either:

- Requiring zero-first updates: only allow `approve(spender, value)` when `value == 0` or the current allowance is zero; otherwise revert. This prevents racing an in-flight change; or
- Encouraging `increaseAllowance`/`decreaseAllowance` and overriding `approve` to enforce zero-first semantics, while updating client applications and integrations to use the incremental functions.

Actionable example for this contract:

- In `contracts/modules/2_CMTATBaseAllowlist.sol` function `approve`, add a check similar to `require(value == 0 || allowance(_msgSender(), spender) == 0, "NON_ZERO_TO_NON_ZERO");` before delegating to the underlying `ERC20Upgradeable.approve`.
- Alternatively, re-route to or expose `increaseAllowance`/`decreaseAllowance` and deprecate direct use of `approve` in integrations and UI.

[Go back to Findings Summary]

I1: Documentation mismatch: `_authorizeSelfBurn` comment references `BURNER_FROM_ROLE` but code enforces `BURNER_SELF_ROLE`

Impact:	Info	Confidence:	High
Target:	contracts/modules/4_CMTATBaseERC20CrossChain.sol	Type:	Code quality

Description

The inline documentation above `CMTATBaseERC20CrossChain._authorizeSelfBurn` incorrectly states that the caller must have the `BURNER_FROM_ROLE`. However, the implementation actually enforces `BURNER_SELF_ROLE` via `onlyRole(BURNER_SELF_ROLE)`.

This inconsistency can mislead integrators and auditors, and may result in incorrect assumptions during operational runbooks or access reviews. While not directly exploitable on-chain, it increases the risk of misconfiguration or incorrect privilege assignment in off-chain processes relying on the documentation.

Listing 8. Code snippet from contracts/modules/4_CMTATBaseERC20CrossChain.sol

```
    /**

     * @custom:access-control
     * - the caller must have the `BURNER_FROM_ROLE`.
     * - We don't allow token holder to burn their own tokens if they don't have this role.
     */

    function _authorizeSelfBurn()
        internal
        virtual
        override(ERC20CrossChainModule)
```

```
        onlyRole(BURNER_SELF_ROLE)
        whenNotPaused
    {}
```

Exploit Scenario

Inaccurate documentation can lead to operational errors and failed workflows.

Consider the following attack scenario:

1. Alice, an operator, provisions users with `BURNER_FROM_ROLE` based on the comment, expecting them to self-burn;
2. Alice instructs Bob (a user) to self-burn, but Bob calls into the self-burn path that requires `BURNER_SELF_ROLE` and the transaction reverts;
3. The team, believing the role is correct per docs, attempts unsafe workarounds or temporarily elevates permissions; and
4. Bob's application suffers disruption or unintended access changes due to the mismatch, increasing operational risk.

Recommendation

Align the documentation with the enforced role and add automated checks to prevent drift.

Recommended approach:

- Fix the comment above `_authorizeSelfBurn` to reference `BURNER_SELF_ROLE`;
- Add tests that assert the self-burn path is gated by `BURNER_SELF_ROLE` and that `BURNER_FROM_ROLE` is insufficient; and
- Consider emitting clear events or using NatSpec `@dev` annotations to prevent confusion in future audits and integrations.

Go back to Findings Summary

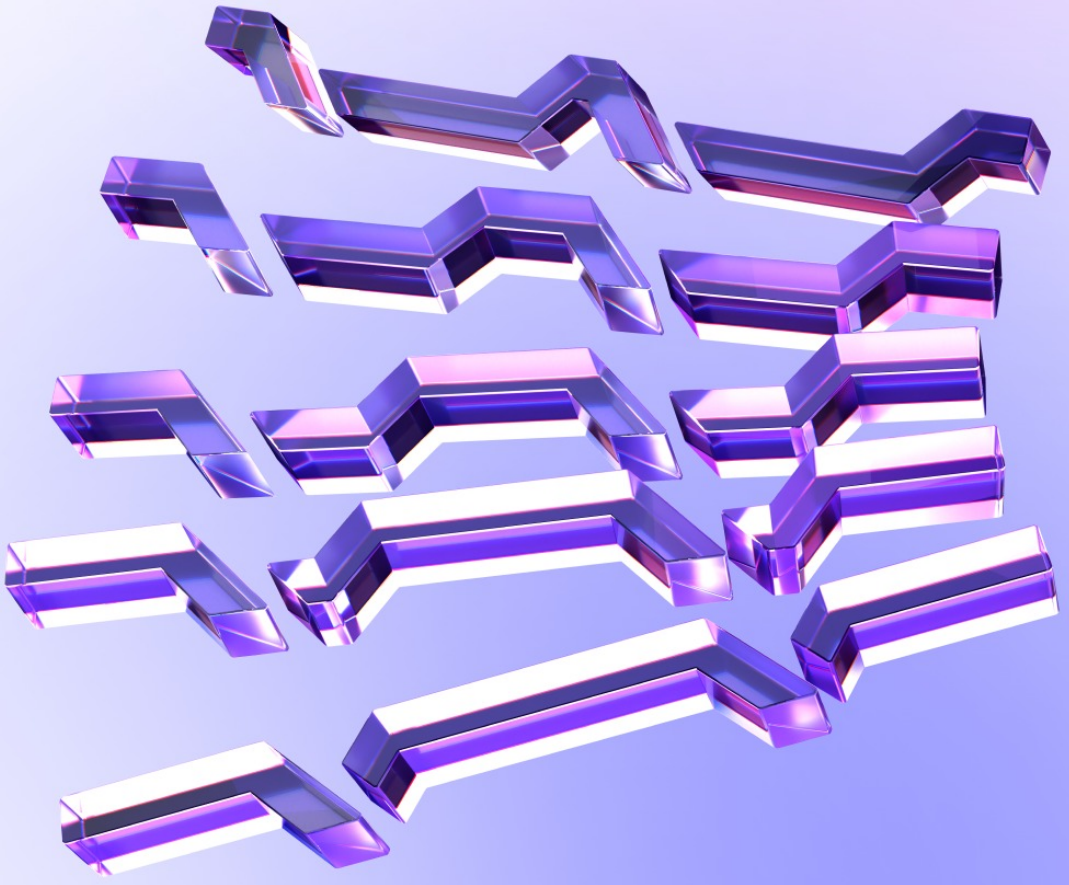
Appendix A: How to cite

Please cite this document as:

Ackee Blockchain Security, Wake Arena AI Report | CMTA: CMTAT, February 10, 2026 12:24 UTC.

Thank You

Ackee Blockchain a.s.

Rohanske nabrezi 717/4
186 00 Prague
Czech Republic

hello@ackee.xyz